

Apéndice A

Manuales técnicos

En este capítulo se incluye el manual técnico que proporciona una guía detallada para el desarrollo, configuración, mantenimiento y despliegue de la aplicación UniLink.

A.1. Requisitos Previos

A.1.1. Herramientas Necesarias

- Android Studio: esta aplicación fue desarrollada en Android Studio Iguana — 2023.2.1 Patch 1, que requiere una versión de AGP (Android Gradle Plugin) de entre 3.2 y 8.3 que se instalará automáticamente con el IDE.
- Git: para control de versiones y acceder al código. Se ha empleado la versión 2.44.0.
- Kotlin: Integrado en Android Studio.
- Firebase Console: Configuración de servicios Firebase.

A.1.2. Instalación de Android Studio

- Descargar e instalar Android Studio.
- Durante la instalación, asegurarse de instalar el SDK de Android correspondiente a la API 28 o superior (Android 9 Pie).

A.1.3. Configuración del proyecto

- Clonar el repositorio de *UniLink* desde Github:
git clone <https://github.com/marcosdeuna/UniLink.git>

- Abrir Android Studio y seleccionar Open.
- Navegar hasta la carpeta del proyecto clonado y seleccionarla.
- Android Studio sincronizará las dependencias del proyecto automáticamente y al terminar se podrá ejecutar la aplicación.

A.2. Configuración de Firebase

En caso de querer acceder a la base de datos y recursos que está utilizando la aplicación de Firebase, contactar con soporte para solicitar un acceso al proyecto y poder incluir colaboradores.

De querer utilizar otro proyecto de Firebase:

- Crear un proyecto en Firebase Console.
- Registrar la aplicación con su ‘applicationId’
- Descargar el archivo ‘google-services.json’ y colocarlo en la carpeta ‘app’ del proyecto.
- Configurar los servicios necesarios como Firestore, Auth, Storage y Messaging.

A.3. Estructura del Proyecto

Como ya se mencionó en el capítulo de diseño, la aplicación sigue el patrón de arquitectura Modelo-Vista-VistaModelo (MVVM), que separa la lógica de negocio de la interfaz de usuario. Esto facilita la modularidad y el mantenimiento del código.

A.3.1. Paquetes Principales

La estructura del proyecto *UniLink* se divide en varios paquetes y recursos organizados de la siguiente manera:

- manifests/AndroidManifest.xml: Archivo de configuración principal de la aplicación.
- kotlin+java/com/marcosdeuna/unilink: Paquete principal del código fuente de la aplicación.
 - data/: Contiene los modelos de datos y las interfaces e implementaciones de todos los repositorios. Son los encargados de gestionar las llamadas a la base de datos de Firebase.

- di/: Contiene los módulos de inyección de dependencias con Dagger.
 - ui/: Contiene las clases relacionadas con la interfaz de usuario y la lógica de presentación. Estas se dividirán en paquetes que corresponderán en mayor medida a funcionalidades de la aplicación para una mayor organización. Dentro de ellos se almacenarán las clases Kotlin relativas a Fragmentos, donde cada uno se corresponderá con un fragment del res/layout al cual se enlazará mediante Data Binding; ViewModels, donde se tratan los objetos observables (LiveData) que obtienen información de los repositorios y los presentan en la Vista; y, de ser necesario, Adaptadores para la presentación de los elementos.
 - util/: Contiene utilidades y clases auxiliares.
 - MainActivity.kt: Actividad principal de la aplicación que maneja la Splash Screen y la navegación principal por los diferentes fragmentos.
 - MyApplication.kt: Clase de aplicación personalizada para inicializar configuraciones globales.
- Recursos (res): En esta carpeta se almacenan los recursos gráficos y de interfaz de usuario de la aplicación.
- drawable: Contiene imágenes y recursos gráficos.
 - layout: Archivos de diseño de la interfaz de usuario. Aquí se encuentran las ventanas de cada actividad, fragmento, diálogos y elementos de adaptadores.
 - menu: Archivos que especifican los elementos que contendrá un menú (barra de navegación, por ejemplo).
 - mipmap: Iconos de la aplicación.
 - navigation: Gráficos de navegación.
 - values: Archivos de recursos como cadenas de texto, estilos, colores y dimensiones.
 - xml: Archivos de configuración adicionales (root_preferences.xml, por ejemplo, que es el layout correspondiente a los ajustes).

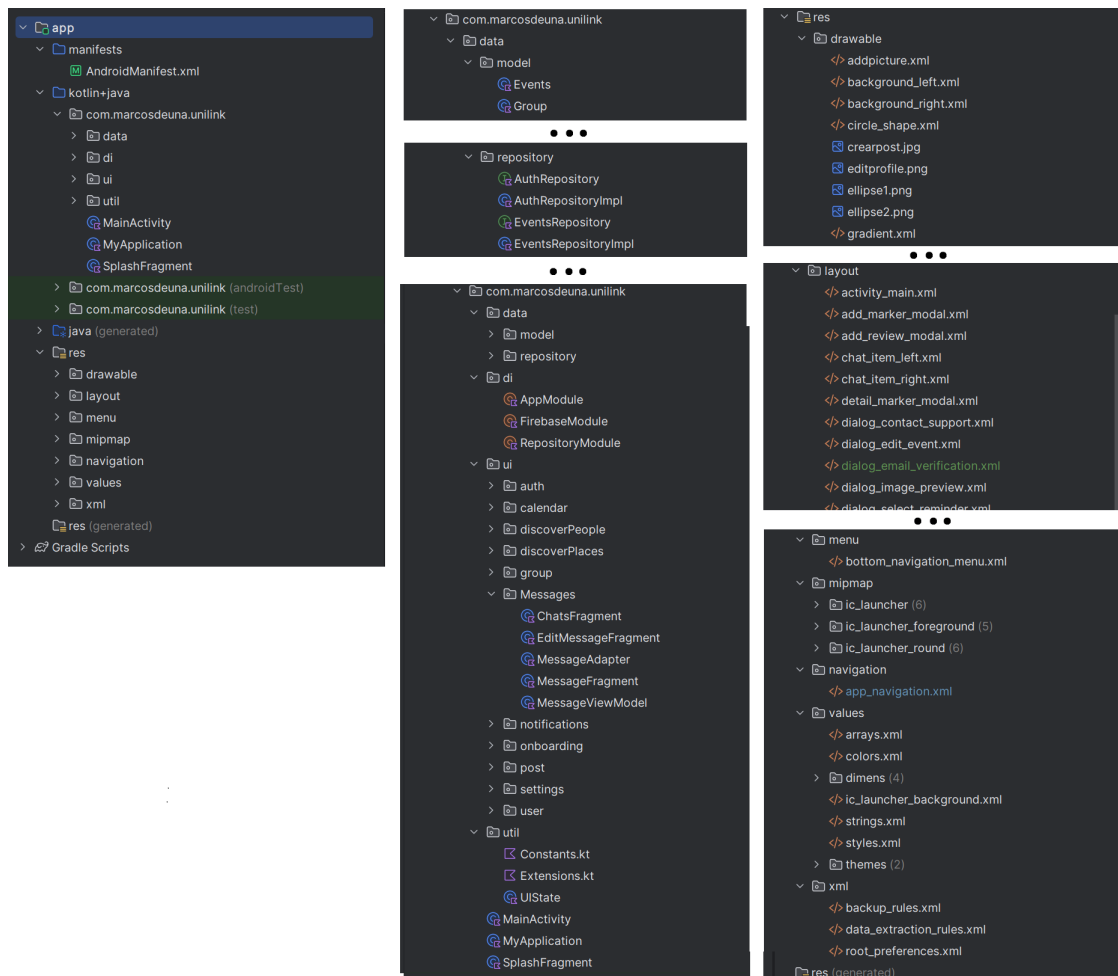


Figura A.1: Estructura de archivos

A.4. Pruebas y Debugging

A.4.1. Pruebas en dispositivos

La aplicación ha sido probada en emuladores configurados con API 28 hasta API 33 para garantizar la compatibilidad:

- Pixel 8 Pro API 33 (Android 13)
- Pixel 4 API 28 (Android 9)

Usuario de prueba:

- Correo: unilink0002@gmail.com
- Contraseña: 123456

A.4.2. Herramientas de Debugging

- Logcat: Para visualizar los logs y errores en tiempo real.
- Debugger de Android Studio: Para establecer puntos de interrupción y analizar el flujo del código.

A.5. Despliegue

Configurar la tarea de generación de APK en Android Studio.

A.6. Mantenimiento y Actualizaciones

A.6.1. Gestión de Versiones

Utiliza Git y GitHub para gestionar el desarrollo de nuevas características, correcciones de errores y actualizaciones de la aplicación de manera estructurada y controlada.

Mantener actualizado el archivo ‘versionCode’ y ‘versionName’ en el ‘build.gradle’.

A.6.2. Actualizaciones de Dependencias

Revisar regularmente las actualizaciones de las dependencias y aplicarlas según sea necesario.

A.7. Contacto y Soporte

Para cualquier problema no resuelto, contactar al equipo de soporte técnico a través del correo ‘unilink0002@gmail.com’.